IndusBrain

The Operating System for Industrial Knowledge

ET AI Hackathon 2026

Problem Statement 8 — AI for Industrial Knowledge Intelligence:
Unified Asset & Operations Brain

Theme: Industrial Intelligence · Document Management · Knowledge Engineering · Quality

Project Submission Document

1. Problem Statement

Reproduced verbatim from the ET AI Hackathon 2026 brief, Problem Statement 8.

Challenge Statement

> *Build an AI-powered Industrial Knowledge Intelligence platform that ingests heterogeneous documents — engineering drawings, maintenance records, safety procedures, inspection reports, operating instructions, project files — across structured and unstructured formats, and makes their collective intelligence queryable, actionable, and continuously updated at the point of need, across any device or function.*

Problem Context (from the brief)

> *A 2024 McKinsey global survey found that professionals in asset-intensive industries spend an average of 35% of their working hours searching for information, clarifying instructions, or recreating documents that already exist somewhere in the organisation. In India specifically, a NASSCOM-EY study of manufacturing and energy companies found that the average large plant operates across 7 to 12 disconnected document systems — P&IDs and engineering drawings in one place, maintenance work orders in another, operating procedures in a third, inspection records in a fourth, and regulatory submissions scattered across email archives. BIS Research estimated that this fragmentation contributes to 18–22% of unplanned downtime events in Indian heavy industry, as maintenance teams make decisions without complete equipment history or failure pattern context. Then there is the knowledge cliff: an estimated 25% of India's experienced industrial engineers and operators will retire within the next decade, taking decades of undocumented operational knowledge with them. Once gone, it cannot be recovered. Knowledge fragmentation in industrial operations is not a file management problem. It is a safety problem, a quality problem, and an operational efficiency problem — and it compounds over time. The organisations that solve it first will have a structural advantage in how they operate, maintain, and improve their assets.*

2. Problem Analysis

Breaking the brief down into the specific failures a solution must address:

Knowledge fragmentation

A single question — "what pressure can this pump run at?" — may require the OEM manual, a later engineering revision notice, the maintenance log and the relief valve register. Those four documents typically live in four different systems, none of which know the others exist. The engineer becomes the integration layer, and the quality of the answer depends entirely on which documents that individual happens to know about.

Silent contradiction

When an operating limit is revised, the original document is rarely withdrawn. Both versions remain in circulation. Two engineers can read two controlled documents, reach opposite conclusions, and both be acting in good faith. Nothing in a conventional document system detects that the two disagree.

Knowledge loss on retirement

The most valuable operational knowledge is often the reasoning behind a decision, not the decision itself: why a particular seal plan was chosen, which failure mode recurs, what the last team learned the hard way. When an experienced engineer leaves, that context leaves too. What remains is a record of what was done, without the understanding of why.

Search that does not understand equipment

Keyword search treats "P-101A" as a string. It cannot know that P-101A is a pump, that P-101B is its standby, that PSV-1201 protects its discharge header, or that a maintenance entry about a seal failure is relevant to a question about pressure limits. Without a model of the equipment itself, search returns documents rather than answers.

Trust

An AI assistant that answers confidently but occasionally invents a number is unusable in a safety context. If an engineer must verify every answer from source, the assistant has saved nothing. Trust is therefore not a nice-to-have feature — it is the precondition for the product being used at all.

3. Our Solution

IndusBrain is an Industrial Knowledge Intelligence platform that converts a fragmented document corpus into a single queryable knowledge base with a traceable reasoning layer on top.

It ingests industrial PDFs, extracts their text (falling back to OCR for scanned pages), recognises the equipment referenced inside them, and builds a knowledge graph linking every asset to every passage that mentions it. An engineer then asks a question in plain English. The system searches both the semantic vector index and the knowledge graph, fuses and re-ranks the results, routes the question to a specialist agent, and returns an answer in which every claim is traceable to the passage it came from.

The governing design principle

> *The system must never invent an answer. In a plant, a confident wrong number about a pressure limit is more dangerous than no answer at all.*

This principle is enforced structurally, not by prompt instruction alone:

- Confidence is computed from measured retrieval quality — the cross-encoder score, the number of independent corroborating sources, and whether the knowledge graph confirmed the passage. It is capped at 95%, because no retrieval system can honestly claim certainty.
- Citations are the passages actually used. When nothing is retrieved, the citation list is empty and the system says so explicitly.
- Dates are never estimated. Maintenance history reports a date only when the source document literally contains one; otherwise it states "date not recorded".
- The decision trace is written by each pipeline node as it executes, not reconstructed afterwards.
- Off-domain questions return 0% confidence and zero citations rather than a plausible-sounding fabrication.

What this looks like in practice

Asked "What is the maximum discharge pressure for P-101A?", IndusBrain does not simply return the highest-ranked passage. It reports that the OEM manual states 285 PSI and a later revision notice states 340 PSI, lists the four mandatory preconditions the revision attaches to the higher figure, identifies from the maintenance log that relief valve PSV-1201 was never reset as required, and concludes that the lower 285 PSI limit therefore still applies.

That answer required reasoning across three documents that no individual had read together. It is the core demonstration of the product.

4. Features Implemented

Core intelligence

Feature	Description
Intelligent Document Ingestion	Three-stage extraction cascade with bounded concurrency; real server-side progress reporting; per-document retry without re-upload.
OCR Pipeline	Scanned pages rasterised at 150 DPI and processed with Tesseract when no text layer exists. Binary resolved centrally across platforms.
Knowledge Graph	Neo4j graph of Document → Chunk → Asset / Standard. Supports both descriptive (PUMP-101) and ISA process (P-101A) tagging conventions.
Hybrid GraphRAG	Parallel vector and graph retrieval, reciprocal rank fusion, cross-encoder re-ranking, and parent-context expansion.
Multi-Agent AI Workflow	LangGraph state machine: supervisor plus four genuinely differentiated specialist agents and a compiler node.
Knowledge Conflict Detection	Deterministic cross-document contradiction detection with parameter-qualifier matching to eliminate false positives.
Organizational Memory	Past maintenance events quoted verbatim with real dates and work-order numbers extracted from the source.
Decision Replay	Node-by-node execution trace with real measurements, written by the pipeline as it runs.
AI Confidence Engine	Evidence-strength score derived from measured retrieval quality, with a published component breakdown.
Explainable AI	Every answer exposes its sources, relevance scores, retrieval origin and full execution trace.

Workspace and interface

Feature	Description
Expert Knowledge Copilot	Streaming chat with live multi-agent pipeline stages and an evidence panel.

Feature	Description
Knowledge Explorer	Every discovered asset with per-category documentation coverage and identified gaps.
Analytics Dashboard	Knowledge health scoring with the calculation formula published in the response.
Impact Simulator	Evidence-based assessment of a proposed operational decision; refuses without evidence.
Document Viewer	In-app PDF rendering plus the extracted passages exactly as indexed, with search.
Conversation History	Titled, searchable threads with evidence preserved across reloads.
Dynamic Suggestions	Starter questions generated from the live corpus; never hardcoded.
Conflict-to-Chat Workflow	One click sends a fully-formed conflict analysis question into a new conversation.
Role-Based Access Control	Manager and Engineer roles, enforced server-side on every endpoint.

5. System Architecture

End-to-end flow



Multi-agent workflow

Implemented as a LangGraph state machine with six nodes. A supervisor performs deterministic intent classification and dispatches to one specialist, which reports back to a compiler node.

```
                  SUPERVISOR
          (records real keyword counts)
                       |
   +---------+--------+--------+---------+
   v         v                 v         v
KNOWLEDGE  RELIABILITY   COMPLIANCE   CONFLICT
specs &    maintenance   safety &     cross-document
manuals    & failures    regulatory   comparison
   +---------+--------+--------+---------+
                      v
                  COMPILER
```

The three retrieval specialists are genuinely differentiated rather than three labels on one code path. Each carries a domain directive appended to the generation prompt, and a domain vocabulary used to re-order passages of comparable relevance. The vocabulary affects ordering only — reported confidence remains the raw cross-encoder score, so domain preference cannot inflate the honesty metric.

The Conflict Agent is structurally different: it runs a deterministic comparison engine rather than retrieval-augmented generation, because no single passage can state that two documents disagree.

Knowledge graph schema

```
(:Document {id, filename})
      | HAS_CHUNK
      v
(:Chunk {uid, index, content, page})
      | MENTIONS              | REFERENCES_COMPLIANCE
      v                       v
(:Asset {tag, type})     (:Standard {code})
```

6. Technologies Used

Frontend

Technology	Version	Purpose
React	19.2	UI framework
TypeScript	6.0	Static typing across the client
Vite	8.1	Build tool and development server
Tailwind CSS	4.3	Styling through a design-token system
React Router	7.18	Client-side routing
markdown-to-jsx	9.8	Assistant answer rendering

Backend

Technology	Version	Purpose
Python	3.11	Runtime
FastAPI	0.111	Asynchronous API framework
Uvicorn	0.30	ASGI server
SQLAlchemy	2.0	Asynchronous ORM
asyncpg	0.31	PostgreSQL driver
Pydantic	2.7	Request and response contracts
bcrypt	—	Password hashing

Artificial intelligence

Technology	Version	Purpose
LangGraph	0.0.69	Multi-agent state machine

Technology	Version	Purpose
Ollama + qwen2.5:7b	—	Local LLM inference (no cloud, no API keys)
sentence-transformers	3.0	Embeddings — all-MiniLM-L6-v2
BAAI/bge-reranker-base	—	Cross-encoder relevance re-ranking

Databases

Technology	Version	Purpose
PostgreSQL	16-alpine	Documents, passages, users, conversations
Neo4j	5.19-community	Asset knowledge graph
ChromaDB	0.5.0	Vector index

Document processing

Technology	Version	Purpose
PyMuPDF	1.28	Primary PDF text extraction and rasterisation
pdfplumber	0.11	Secondary extraction for awkward layouts
pytesseract / Tesseract	0.3.10	OCR for scanned pages

Infrastructure and tooling

Category	Tools
Deployment	Docker Compose (PostgreSQL, Neo4j, ChromaDB)
Development	Git, ESLint, TypeScript compiler, Vite
Testing	Custom end-to-end smoke suite (43 assertions), pipeline diagnostics
Networking	httpx (async HTTP), anyio (thread offloading), Server-Sent Events

7. Expected Deliverables

Mapping the submission against the deliverables listed in the problem statement.

Required deliverable	How IndusBrain satisfies it
Working Prototype	Fully functional local deployment. 13-document refinery corpus indexed across three databases. All features operational and verified by a 43-assertion automated end-to-end suite passing 43/43.
Architecture Diagram	Included in Section 5 of this document and in the repository README: system flow, multi-agent workflow, and knowledge graph schema.
Presentation Deck	Prepared separately. This submission document provides the complete narrative, evidence and technical detail it draws on.
Demo Video	A 3–4 minute recorded walkthrough. A scene-by-scene script and a verified prompt set are included in the Submission folder of the repository.

Evaluation focus

The brief specifies particular evaluation criteria. Our position against each:

Evaluation focus	Our result
Entity extraction accuracy across document types	22 equipment assets extracted from 13 documents with zero false positives. Document references (ERN-2024-0342, WO-2024-0655), part numbers (JC-2800-65) and register identifiers (INS-PSV-2024) are explicitly excluded from the asset register.
Query answer quality on domain-expert questions	Verified across 11 benchmark prompts. Evidence strength ranges from 18% to 91%, correctly reflecting how completely each question is supported — not uniformly high.
Knowledge graph linkage completeness	13 Document nodes, 67 Chunk nodes, 22 Asset nodes, 1 Standard node and 197 relationships. P-101A alone links across 11 of the 13 documents.
Time-to-answer versus traditional search	Retrieval-only mode returns cited passages in under one second. Full AI reasoning takes 40–90 seconds on CPU. Both replace a manual search across multiple systems.

Evaluation focus	Our result
Compliance gap detection accuracy	19 documentation gaps identified across four evidence categories, each naming the specific asset and the missing category.
Cross-functional knowledge discovery	Demonstrated by the P-101A case: an answer that requires the OEM manual, the engineering revision notice and the maintenance log to be reasoned over together.
Validated with real industrial document samples	A realistic refinery Crude Distillation Unit corpus following ISA tagging conventions, containing genuine cross-document situations rather than synthetic filler.

8. Judging Criteria

Criterion	Weight
Innovation	25%
Business Impact	25%
Technical Excellence	20%
Scalability	15%
User Experience	15%

Innovation – 25%

Three elements distinguish IndusBrain from a conventional RAG implementation.

Hybrid GraphRAG with corroboration as an evidence class

Vector similarity and knowledge-graph traversal run in parallel, and a passage retrieved independently by both is marked as corroborated. This is not merely a scoring bonus: a graph-corroborated passage is admitted as evidence even when its semantic similarity score falls below the acceptance threshold, because an exact equipment identifier match is a fact rather than a similarity estimate. This directly solves a real failure mode in which a correct but weakly-worded passage would otherwise be discarded.

Deterministic conflict detection as a distinct agent

No single passage can state that two documents contradict one another, so retrieval alone cannot find contradictions. IndusBrain therefore runs a separate deterministic engine that binds measurements to assets at sentence level and compares only measurements sharing the same parameter qualifier — preventing a pump's suction pressure from being reported as contradicting a casing test pressure merely because both are expressed in PSI.

Verifiable honesty as an architectural constraint

Refusal is treated as correct behaviour rather than failure. The confidence score is a published formula over measured quantities, the decision trace is written during execution, and the impact assessment declines to produce engineering figures it cannot evidence. We consider this the most genuinely novel aspect of the work: most AI assistants optimise for appearing helpful, whereas a safety-critical industrial assistant must optimise for being trustworthy.

Business Impact — 25%

The brief quantifies the problem; IndusBrain addresses each element directly.

Problem from the brief	How IndusBrain addresses it
35% of working hours spent searching	A question spanning four documents is answered in seconds with citations, replacing a manual search across several disconnected systems.
7–12 disconnected document systems	A single ingestion pipeline and one queryable knowledge graph across all document types.
18–22% of unplanned downtime from fragmentation	Organizational Memory surfaces repeat failures and their root causes; conflict detection prevents equipment being operated against a superseded limit.
25% of engineers retiring within a decade	Institutional knowledge is captured in a queryable graph. Reasoning that previously existed only in an individual's memory becomes an organisational asset.

Operational value

- Faster decisions at the point of need, with the source available for verification.
- Fewer incidents caused by acting on superseded documentation.
- Compliance evidence assembled from the corpus rather than reconstructed before an audit.
- Documentation gaps made visible per asset, so remediation can be prioritised.

- Onboarding accelerated — a new engineer can query decades of history immediately.

Technical Excellence — 20%

- Asynchronous FastAPI backend with non-blocking inference and bounded ingestion concurrency.
- LangGraph state machine with typed state channels and append-style reducers.
- Parent-child retrieval: search matches the precise child passage while the model receives the larger parent block for context.
- Embedding consistency — the same model is used at index time and query time, avoiding the silent recall degradation caused by mismatched embedders.
- Every asset-tag definition centralised in one module, so no component can recognise equipment that another component cannot.
- Graceful degradation — datastore connections are lazy and retried, so the API starts and operates even when a backing service is unavailable.
- Referential integrity across three stores: deleting a document removes its Postgres rows, its Chroma vectors, its Neo4j nodes, and any asset node left orphaned.
- 43-assertion automated end-to-end suite, plus per-stage diagnostics for ingestion and retrieval.
- Zero TypeScript errors, zero ESLint errors, clean production build.

Scalability — 15%

Detailed in Section 9.

User Experience — 15%

- One coherent design system: a single accent colour, semantic colours reserved for meaning, monospace reserved for data such as asset tags and filenames.
- Live pipeline visibility — the user sees classification, agent dispatch, retrieval, ranking and generation as they happen, rather than an unexplained wait.
- Honest empty states — every empty view explains why it is empty and what to do next.
- Progressive disclosure — the answer is primary; evidence, confidence, history and the execution trace are one click away.
- Accessibility — visible focus indicators, ARIA roles on tabs and radio groups, and full respect for prefers-reduced-motion. Severity is never conveyed by motion alone.

- Responsive from mobile to desktop, addressing the brief's requirement that field technicians are served as well as desk-based engineers.
- Error containment — a failure in one panel cannot blank the application.

9. Scalability

Current architecture

Dimension	Current position
Documents	Verified with 13 documents / 67 passages. The architecture holds to roughly 10,000 documents on a single node; ChromaDB and Neo4j both handle that volume comfortably.
Concurrency	Ingestion is explicitly bounded — 8 extraction threads per document, 2 documents concurrently — so a large batch upload degrades throughput rather than exhausting memory.
Query load	Stateless API. Dashboard responses are cached with a short TTL, and the frontend suspends polling when the browser tab is hidden.
Storage	Postgres is the source of truth. The vector index and knowledge graph are both derivable, and either can be rebuilt from Postgres without re-uploading or re-OCRing anything.

Path to enterprise scale

Horizontal API scaling

The API holds no per-request state, so it scales horizontally behind a load balancer immediately. Ingestion would move from in-process background tasks to a dedicated worker queue (Celery or RQ), allowing extraction capacity to scale independently of request-serving capacity.

Managed data services

ChromaDB would be replaced by a managed vector service (Pinecone, Weaviate, or pgvector for a consolidated deployment) and Neo4j by Neo4j Aura. Both are interface-compatible with the existing service abstractions, so the change is confined to two modules.

Inference scaling

Local Ollama inference is the current throughput bottleneck at 40–90 seconds per answer on CPU. Production deployment would use GPU-backed inference or a hosted model endpoint behind the existing gateway, which already reports which engine produced each answer.

Multi-plant and multi-tenant

The knowledge graph naturally partitions by plant: adding a Plant node above Document, and a tenant column on the Postgres tables, gives isolation without restructuring. Cross-plant federation then becomes a deliberate query-time choice — a failure at one site can warn every comparable asset across the organisation.

Departmental scale

The existing RBAC permission table extends to additional roles — inspector, contractor, auditor — by adding permission sets. Navigation and page access are already derived from the server's permission table rather than hardcoded in the client, so new roles require no frontend changes.

Future SaaS architecture

A managed offering would add tenant isolation at the storage layer, per-tenant embedding namespaces, usage metering, SSO integration, and audit logging. The current separation between ingestion, retrieval and reasoning already maps cleanly onto independently deployable services.

10. Current Limitations

Stated plainly. A system that claims more than it delivers is precisely the problem this product exists to solve.

Demonstration Mode authentication

Full authentication is implemented and working: bcrypt password hashing with a tunable work factor, server-side revocable session tokens (chosen over stateless JWTs specifically so that logging out genuinely invalidates access), protected routes, and privilege-escalation prevention on registration.

For this submission the platform runs with AUTH_MODE set to demo, which allows judges to switch between the Engineer and Manager experiences instantly without creating accounts or repeatedly signing in. This was a deliberate decision to keep evaluation focused on the AI, GraphRAG, multi-agent architecture and knowledge intelligence capabilities rather than on account management.

Importantly, role permissions remain fully enforced server-side in both modes. An Engineer receives HTTP 403 when attempting to upload or delete a document regardless of what the interface displays. Only identity verification is relaxed; authorisation is real. Switching to production authentication is a single configuration change (AUTH_MODE=secure), and a production deployment would additionally integrate enterprise SSO or LDAP with audit logging.

Local inference latency

Running qwen2.5:7b locally on CPU takes 40–90 seconds per answer. This was a deliberate trade: local inference means no API keys, no per-query cost, and no plant data leaving the network — all genuine requirements in industrial environments. A retrieval-only mode returns cited passages instantly for latency-sensitive use.

Document format coverage

Ingestion currently handles PDFs, including scanned documents through OCR. Spreadsheets, email archives and CAD or P&ID drawings are not yet parsed. P&ID parsing in particular would require computer vision and is identified as a future enhancement.

Conflict detection scope

Detection covers numeric parameter contradictions in recognised engineering units. Procedural contradictions — two documents prescribing incompatible sequences — are not detected, as these require semantic rather than numeric comparison.

Single-tenant deployment

The current build serves one plant. Multi-plant partitioning is architecturally straightforward but is not implemented.

11. Future Enhancements

Enhancement	Description
Industrial Knowledge DNA	A per-asset knowledge fingerprint tracked over time, so documentation degradation becomes visible before it contributes to an incident rather than after.
AI Curiosity Engine	Proactively identifies questions the corpus cannot currently answer and requests the specific missing documents, turning the system from reactive to actively self-improving.
Autonomous Knowledge Improvement	Detects broken cross-references, missing revision control and structural gaps, then generates remediation tasks for the knowledge manager.
Predictive Maintenance Agent	Correlates failure history with operating conditions and inspection findings to forecast likely next failures and recommend intervention timing.
Impact Simulator 2.0	Extends the current evidence-based assessment with equipment physics models and historical failure rates, enabling quantified rather than purely documentary what-if analysis.
Digital Twin Integration	Binds knowledge-graph assets to a live plant model, enabling spatial reasoning such as identifying which equipment is affected by an isolation.
IoT and SCADA Telemetry	Combines live sensor readings with documented limits, so an excursion beyond a documented threshold is flagged in real time against the governing document.
P&ID Computer Vision	Parses engineering drawings to extract equipment tags and process connectivity automatically, substantially enriching the knowledge graph.
Enterprise Authentication	SSO and LDAP integration with full audit logging, replacing demonstration mode.
Multi-Plant Knowledge Sharing	Federated knowledge graphs so a failure at one site proactively warns every comparable asset across the organisation.
Knowledge Evolution Engine	Tracks how a procedure changed across revisions and why, preserving the reasoning behind changes rather than only the current state.
Cloud and SaaS Deployment	Managed vector and graph services, horizontal API scaling, tenant isolation and usage metering.

12. Verification and Testing

Every claim in this document was verified against the running system rather than inferred from source code.

Automated test results

Check	Result
End-to-end smoke suite	43 assertions passed, 0 failed
Demo prompt verification	11 of 11 prompts behave as expected
TypeScript compilation	0 errors
ESLint	0 errors, 5 informational warnings
Production build	Succeeds

Live system state at submission

Component	State
Docker services	PostgreSQL, Neo4j and ChromaDB all running
Documents indexed	13, all EMBEDDED, zero failures
Passages	67 in PostgreSQL, 67 vectors in ChromaDB
Knowledge graph	13 Document, 67 Chunk, 22 Asset, 1 Standard — 197 relationships
Conflicts detected	1 genuine contradiction, zero false positives
Documentation gaps	19 identified across four evidence categories

Honesty verification

The following behaviours were explicitly tested, because they are the ones most easily faked:

Test	Result
Off-domain question ("What is quantum computing?")	0% confidence, 0 citations, explicit refusal
Single-character input ("A")	Rejected by the input guard before retrieval runs
Nonsense simulator input ("adas")	Refuses; returns no engineering metrics whatsoever
Conflict detection with no contradiction present	Returns empty with a stated reason
Deleting all documents	All four stores emptied; suggestions and assets disappear
Engineer attempting upload or delete	HTTP 403 from the API, independent of the interface

13. Conclusion

IndusBrain addresses the problem the brief describes: industrial knowledge fragmented across disconnected systems, contradictions that no system detects, and expertise that disappears when people retire.

The platform ingests heterogeneous industrial documents, builds a knowledge graph of the equipment described within them, and answers engineering questions through a multi-agent reasoning pipeline that combines semantic and structural retrieval. Every answer carries its sources, a measured confidence score, and a complete execution trace.

The decision that shaped the project most was the refusal to fabricate. Earlier in development the system reported constant confidence values, invented maintenance dates, generated citations when nothing had been retrieved, and produced engineering figures for nonsense input. Every one of those behaviours was removed and replaced with measured values or an explicit refusal.

That work is not visible as a feature. It is visible as an absence — the absence of answers the system cannot support. In a plant, where a wrong pressure limit can injure someone, we consider that absence the most important thing we built.

Knowledge that outlives the people who created it.